# Generative AI Notebook Authoring Guide

**Self-link**: go/genai-notebook-authoring-guide
**Authors**: Eric Dong
**Contributors**:: Polong Lin Lavi Nigam Holt Skinner
**Updated**: Jan 29, 2026
**Visibility**: Google Internal

This document provides a simplified process for Google contributors to commit their notebook samples to Generative AI sample Github repository GoogleCloudPlatform/generative-ai.

This tutorial assumes that you have basic knowledge of git operations. If you don't, it's OK. You can learn as you follow this guide.

If you have any questions, please feel free to comment directly in this doc with your questions.

# Before you begin

These steps are the one-time setup to prepare your access and environment.

## Getting access

1. Register your GitHub account at go/github.
2. Join the GoogleCloudPlatform organization
    a. Go to https://opensource.corp.google.com/github/orgs/join?join=googlecloudplatform and request to join the GoogleCloudPlatform organization. Check the box "I need write access on a public repository".
    b. You'll receive an email to your GitHub registered email to approve the request to join. Approve it.
3. Request to join `generative-ai-samples-contributors` group from: https://github.com/orgs/GoogleCloudPlatform/teams/generative-ai-samples-contributors/members

## Setting up SSH

1. If you haven't already set up SSH authentication to GitHub, follow this instruction: Adding a new SSH key to your GitHub account. Or run these commands to create a SSH key:

```
git config --global user.name "<Your Name>"
git config --global user.email "<yourldap@google.com>"
ssh-keygen -t ed25519 -C "<yourldap@google.com>"
```

Notes:

- If your Github account is not associated with your Google account then <yourldap@google.com> should be the email address associated with your Github account.

**Example** output:

```
$ git config --global user.name "Eric Dong"
$ git config --global user.email "itseric@google.com"
$ ssh-keygen -t ed25519 -C "itseric@google.com"
        Generating public/private ed25519 key pair.
        Enter file in which to save the key (/Users/itseric/.ssh/id_ed25519):
        Created directory '/Users/itseric/.ssh'.
        Enter passphrase for "/Users/itseric/.ssh/id_ed25519" (empty for no passphrase):
        Enter same passphrase again:
        Your identification has been saved in /Users/itseric/.ssh/id_ed25519
        Your public key has been saved in /Users/itseric/.ssh/id_ed25519.pub
        …
        +----[SHA256]-----+
```

2. Then you can print your **public key** by using a command like:

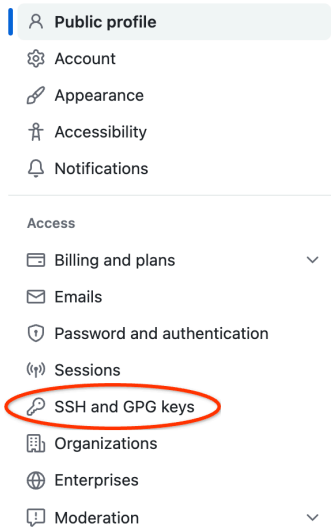
```
cat /home/jupyter/.ssh/id_ed25519.pub
```

The path may be different in your environment, for example, the path may be like this on MacOS:

/Users/<your-ldap>/.ssh/id_ed25519.pub

For example:
$ cat /Users/itseric/.ssh/id_ed25519

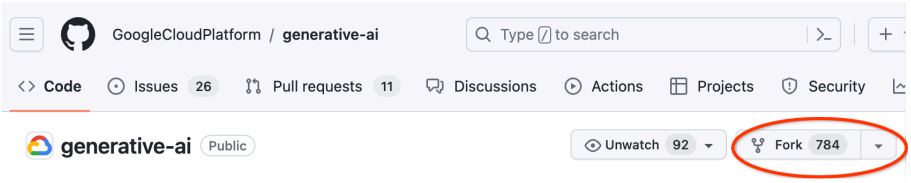
3. And then copy the printed **public key** to your git profile “Settings > SSH and GPG keys > New SSH key”



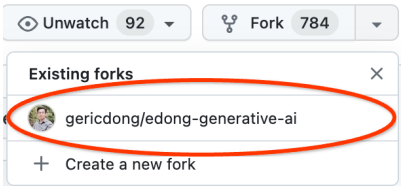

4. Alternatively, you can set up a personal access token for authentication to GitHub, following this instruction: [Creating a personal access token](#).

## Cloning git repo

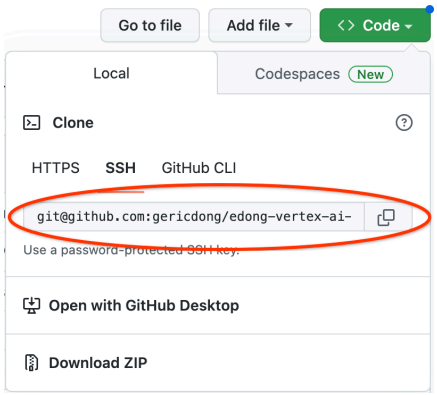
1. Using a web browser, access to git repo generative-ai https://github.com/GoogleCloudPlatform/generative-ai
2. Create a new fork.




3. Select your forked repo.




4. Copy the SSH URL (if using SSH key) or HTTPS URL (if using personal access token) to your forked repo:




5. On your local machine, open a command prompt, clone the forked repo by running `*git clone <SSH/HTTPS>*` command.

```
git clone <SSH/HTTPS>
```

git clone git@github.com:gericdong/edong-generative-ai.git

6. **Change directory to the cloned repo**, update the cloned repo to point to the parent repo

```
git remote add upstream
https://github.com/GoogleCloudPlatform/generative-ai
```

7. Verify that your branch by running command `git remote -v`.

# When you're ready to make code changes

## Creating a new branch

As a best practice, create a new branch each time you're about to begin new code changes.

1. Go to the directory of your cloned repo
2. Update your local main branch:

```
git fetch --all
```

⚠️This is likely the starting point of your regular PR process. Everything above is just a one-time setup.

(In the directory of your cloned repo)

```
git checkout -B main upstream/main
git fetch upstream
git merge upstream/main
```

3. Create a new branch and switch to it:

```
git checkout -b <BRANCH_NAME>
```

   (Or, git checkout <BRANCH_NAME> for existing branch)

   Suggestion: if changes related to bug, replace <BRANCH_NAME> with the number of your tracking bug for example 1234567 so the branch name is b1234567

4. To confirm that you're now in your new branch, you can run:

```
git status
```

## Making code changes

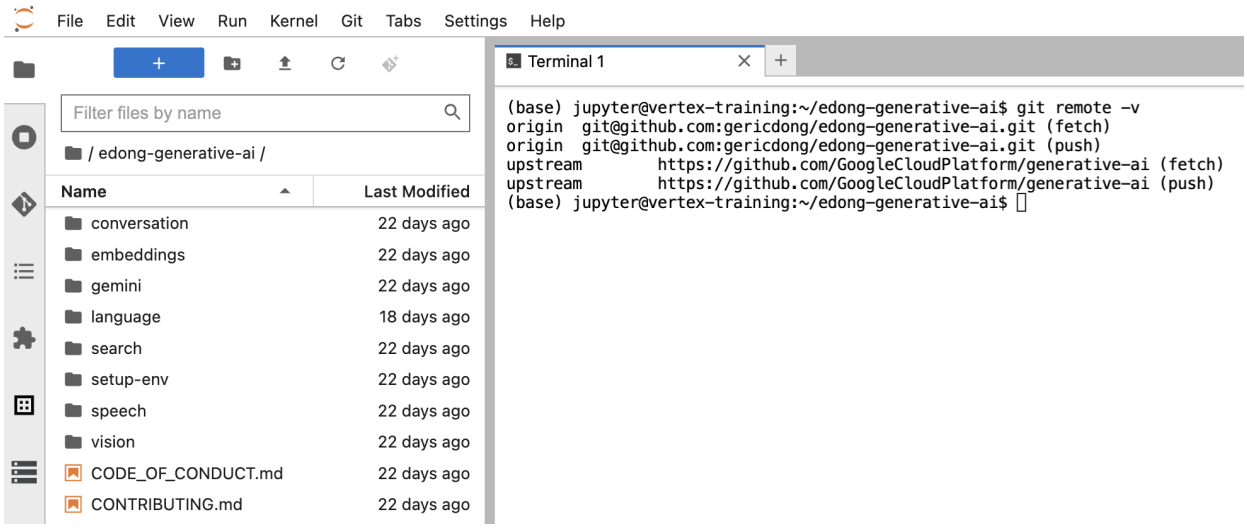
- Use the notebook template if you create a new notebook https://github.com/GoogleCloudPlatform/generative-ai/blob/main/notebook_template.ipynb
- Next, make changes to the notebook(s) and other files in your local cloned repo. You can use Colab or Vertex AI Workbench, VSCode or your favorite editors to make changes.

### Option 1: Vertex AI Workbench Setup instructions

Vertex AI Workbench is recommended for internal users as Workbench offers a more integrated experience. Setup instructions:

- On the Console, open Vertex AI > Workbench
- Create a User-managed Notebooks instance
- Open JupyterLab
- Open Terminal
- Clone your repo (see setup above)
- Use Jupyter Notebook to develop/modify your notebooks

The Workbench environment looks like this:




**Before checking in your code, make sure to**

- ~~Clear all outputs**~~
- Remove hard-coded project ID or variables that you don't want exposed externally

**Outputs: in some cases, you may decide to keep the outputs, which is OK too. Just make sure you check each output if there are things you don't want exposed externally.

## Option 2: VS Code

### Installing Python

Python's installation instructions vary by operating system. Follow the instructions below to install a custom version Python3 and create a virtual environment on MacOS.

[Setting up a Mac development environment with pyenv and pyenv-virtualenv]

These are the commands to creating virtual environments, accessible by running the venv module.

```
cd <Your rep dir>
python3 -m venv .venv
source .venv/bin/activate
```

Typically you will name the venv venv, or venv38 for a python 3.8 venv, for example

python3 -m venv .venv
source .venv/bin/activate

To exit the virtual environment, run

```
deactivate
```

## Formatting your code changes

For any changed notebook, you need to run the linter script *run_linter.sh* before adding the notebook to the commit. There are two ways to run the script:

**Using docker**

1. Start your docker (if you're using Workbench, docker is already running). Check the status of your cloned repo. The command will give you a list of updated files.

```
git status
```

2. Run the following command to reformat the updated notebook files. (make sure your Docker daemon is running in your environment)

```
docker run -v ${PWD}:/setup/app gcr.io/cloud-devrel-public-resources/notebook_linter <your notebook file>
```

For example,
$ docker run -v ${PWD}:/setup/app gcr.io/cloud-devrel-public-resources/notebook_linter gemini/getting-started/intro_gemini_curl.ipynbrl.ipynb

This docker image has all the required packages installed. You run the command each time for one of the updated notebook files.

If there is a lint error, address the issue and run the lint command again. You may need to iterate the process until you see a message as below:

```
All tests finished. Exiting with return code = 0
```

Note: This will autofix most lint issues. If there are remaining unfixed issues, they will be outputted and you will need to manually correct them.

**Or, using commands**

source venv/bin/activate

```
python3 -m pip install -U -r .github/workflows/notebook_linter/requirements.txt
```

```
.github/workflows/notebook_linter/run_linter.sh <your notebook file>
```

Example output:

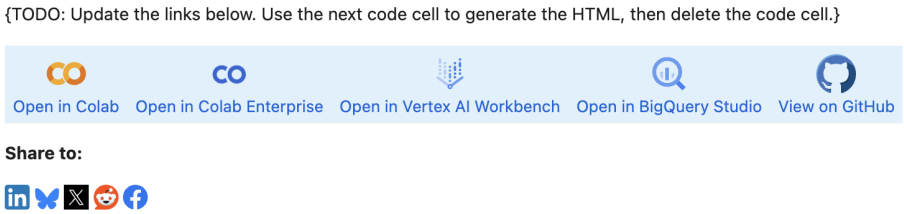
$.github/workflows/notebook_linter/run_linter.sh
gemini/controlled-generation/intro_controlled_generation.ipynb

Test mode: false
Checking notebook: gemini/controlled-generation/intro_controlled_generation.ipynb
Running isort...
Running black...
All done! ✨ 🍰 ✨
…
Success: no issues found in 1 source file
Notebook lint finished with return code = 0

# When you're ready to commit your changes

## Before committing your code, make sure to

- **Clear all outputs.** There are exceptions to this when the output can help the content.
- **Remove hard-coded IDs**
  - Project ID > [your-project-id]
- **Make sure links publicly accessible** (e.g. "https://pantheon.corp.google.com" is internal)
- **Update notebook links**




- **Format your code** with linter (see step above)

## Commit

You're ready to commit and push your changes to GitHub.

*Note, we recommend only one notebook per pull request.*

## Committing and pushing your changes

1. Go to the directory to your cloned repo.
2. Check the status of your cloned repo. The command will give you a list of updated files.

```
git status
```

3. Add your updated files to the staging area. Replace <FILE_NAMES> with the file names of the changed files. Alternatively, you can do a separate `git add` for each individual file changed.

```
git add <FILE_NAMES>
```

   If the PR is to delete a file, use “git rm <FILE_NAMES>”

4. Commit the changes.

```
git commit -m "<commit type>: Your commit comment"
```

The commit type can include the following:

- feat – a new feature is introduced with the changes.
- fix – a bug fix has occurred.
- chore – changes that do not relate to a fix or feature and don't modify code or test files (for example updating dependencies)
- refactor – refactored code that neither fixes a bug nor adds a feature.

5. Push your local changes to the remote repo. Replace <BRANCH_NAME> with your branch name, e.g. b1234567

```
git push origin <BRANCH_NAME>
```

   For a new PR, the push command output prints a message, including a URL. Copy this URL so that you can use it to create a pull request on a web browser. This is an example URL from a push command:

```
remote: Create a pull request for 'genai-update-1218' on GitHub by visiting:
remote:      https://github.com/gericdong/edong-generative-ai/pull/new/genai-update-1218
```

If you are using a personal access token, enter the access token when prompted for username, and hit return on password (no entry).

⚠️When asked for user name, you will need to update your git remote URL, as example below:

$ git remote set-url origin git@github.com:gericdong/edong-generative-ai.git

## Creating a pull request

1. On a browser tab, paste in the URL from the output of the git push command.
2. For the title field, paste in the same message that you used for your commit message.
3. Click Create pull request.
4. Add a summary, where indicated at the top of the edit box.
5. Follow the PR requirements based on whether submitting to community or official

## Iterating your changes

If you need to make changes to your notebook(s) either to satisfy the tests or to address the reviewers' feedback, make changes locally and then go through the step of "Committing and pushing your changes" above to commit the changes. The new changes are included in the existing pull request and will trigger CI tests automatically.

The code review is typically an iterative process.

If the reviewers are satisfied with the changes, they will approve and merge the changes to Github repo.

## Resolve any linting errors

This will ensure that your changes are formatted correctly.

For errors generated by check-spelling, go to the Job Summary to read the errors.

- Fix any spelling errors that are found.
- Forbidden Patterns are defined as regular expressions, you can copy/paste them into many IDEs to find the instances. Example for Visual Studio Code.
- Add false positives to .github/actions/spelling/allow.txt. Be sure to check that it's actually spelled correctly!

# When you need help

- Reach out to members at the Cloud Advocacy and Engineering team or reach out to one of the following on-duty engineers:
    - Polong Lin Holt Skinner Eric Dong Ivan Nardini Erwin Huizenga Katie Nguyen

# Additional resources

- Contribution guide on Github
- Vertex AI notebook authoring guide: go/vertex-notebook-authoring-guide

## FAQs

- **Pull changes from remote branch**

```
None
git pull origin <branch>
```

Add –rebase if you want rebase

- **Your branch and 'upstream/<branch>' have diverged**

```
None
git reset --hard upstream/<branch>
```

Then that just resets your (local) copy of main to the correct point, as represented by (remote) origin/main.

- Undo a commit after push

```
None
git reset --hard HEAD~1
git push -f <remote> <branch>
```

- Undo a commit

```
None
(git commit -m "Something terribly misguided")
git reset HEAD~
```

- Pull the latest from main

```
None
git pull upstream main
```

- “The tip of your current branch is behind its remote counterpart” when git push

```
None
git push -f origin <BRANCH>
```

- ! [rejected]  failed to push some refs to 'github.com:gericdong/edong-generative-ai.git'
  Updates were rejected because the remote contains work that you do not have locally.

```
None
$ git push origin model-update7
To github.com:gericdong/edong-generative-ai.git
 ! [rejected]        model-update7 -> model-update7 (fetch first)
error: failed to push some refs to
'github.com:gericdong/edong-generative-ai.git'
hint: Updates were rejected because the remote contains work that
you do
hint: not have locally. This is usually caused by another
repository pushing
...
$ git pull origin model-update7
...
$ git push origin model-update7
Enumerating o
```

- rejected "non-fast-forward"

```
None
git fetch
git rebase <branch>
git push origin <branch>
```